UNIVERSIDAD ICESI · SCHOOL OF ENGINEERING · DEPARTMENT OF ICT
Discrete Structures III (CyED III, Course 09772) · Group 3

ResumeLens

Formal Language-Based Resume Screening Using Transducers, Automata, and Context-Free Grammars

Nicolás Santiago Buelvas González
Stages 1 & 2: Regex Extraction & FST Normalization
Advisor: Prof. Andrés Aristizábal

Daniel Alexander Nastul Flórez
Stages 3 & 4: DFA Recognition & textX DSL
Advisor: Prof. Andrés Aristizábal

Albert Valencia Escobar
Stage 5: Visualization & Empirical Verification
Advisor: Prof. Andrés Aristizábal

Problem Statement: The Limits of Naive Screening

1. Lexical Fragmentation

Technical skills lack standardized representation across candidate resumes.

- 'PostgreSQL', 'Postgres', 'postgres', 'psql'
- 'React.js', 'ReactJS', 'React', 'react'
- Direct string matching fails on lexical synonyms.
- Surface noise breaks traditional keyword filters.

2. Structural Insensitivity

Keyword counting ignores critical architectural hierarchy and prerequisite stacks.

- Full Stack requires Frontend -> Backend -> DB.
- Keyword bags accept resumes missing core tiers.
- Fails to enforce prerequisite competencies.
- Syntactic sequence is completely discarded.

3. The Black-Box Deficit

Opaque deep learning / LLMs introduce compliance and reproducibility hazards.

- Unexplainable candidate rejections.
- Hallucination and non-deterministic scores.
- Lack of formal mathematical audit trails.
- Regulatory compliance and fairness risks.

RESEARCH OBJECTIVE & FORMAL THESIS

Construct an inspectable, fully deterministic pipeline leveraging Formal Language Theory (Chomsky Hierarchy: Regular Languages, Finite-State Transducers, Deterministic Finite Automata, and Context-Free Grammars) to audit and classify resume qualifications.

System Architecture: End-to-End 5-Stage Pipeline

STAGE 1

Entity Extraction

Python Regex

Raw plain-text curricula parsed for contact info and surface technical keywords.

Input: Raw CV Text
Output: Surface Token Set

$L(R) \subseteq \Sigma_text^*$

STAGE 2

FST Normalization

pyformlang FST

Transduces surface aliases to canonical tokens and applies categorical role sorting.

Input: Surface Tokens
Output: Canonical Sequence

$M = (Q, \Sigma, \Gamma, \delta, \omega, q0, F)$

STAGE 3

Profile Recognition

Deterministic FA

Evaluates sorted canonical sequence against strict multi-tier role automata.

Input: Sorted Sequence
Output: ACCEPT / REJECT

$M = (Q, \Sigma, \delta, q0, F)$

STAGE 4

DSL Specification

textX CFG

Parses and validates candidate data into strongly typed Abstract Syntax Trees.

Input: Candidate Model
Output: Validated AST

$G = (VN, VT, P, S)$

STAGE 5

Artifact Generation

Jinja2 & MD

Produces human-readable visual scorecards, Markdown summaries, and serialized DSL.

Input: Validated AST
Output: Multi-Format Reports

`Traceable Artifacts`

PIPELINE GUARANTEE: DETERMINISM & TRACEABILITY

Every candidate decision is auditable: inputs trace linearly through formal transducers and finite automata. Zero opaque weights, zero stochastic inference.

Stage 1: Contact & Technical Skill Extraction (Regex)

Contact Language Specifications

Email Language (L_email):

```
\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b
```

Phone Language (L_phone):

```
(?:(?:\+|00)\d{1,3}[\s.-]?)?(?:\(?\d{2,4}\)?[\s.-]?)?\d{3,4}[\s.-]?\d{3,4}\b
```

LinkedIn Profile Language:

```
(?:https?:\/\/)?(?:www\.)?linkedin\.com\/in\/([A-Za-z0-9_-]+)\/?
```

GitHub Profile Language:

```
(?:https?:\/\/)?(?:www\.)?github\.com\/([A-Za-z0-9_-]+)\/?
```

Technical Skill Boundary Patterns

Skill Entity Language Definition:

```
For canonical token k, R_k = \b (v1 | v2 | ... | vm) \b
```

JavaScript Surface Pattern:

```
\b(JavaScript|Javascript|JS)\b (case-sensitive)
```

React Surface Pattern:

```
\bReact(?:\.js|JS)?\b (case-insensitive)
```

PostgreSQL Surface Pattern:

```
\bPostgre(?:SQL|s)\b (case-insensitive)
```

Scikit-Learn Surface Pattern:

```
\b(scikit[- ]learn|sklearn)\b (case-insensitive)
```

EXTRACTION SCOPE & DESIGN DECISION

Stage 1 extracts the surface tokens exactly as written (e.g. 'JS', 'React.js', 'Postgres'). It deliberately avoids canonical mapping to keep Stage 1 strictly lexical and decouple extraction from canonical transduction.

Stage 2: Finite-State Transducers for Skill Normalization

Formal 7-Tuple Definition

M = (Q, Σ, Γ, δ, ω, q0, F)

Q: Finite set of states tracking consumed character prefixes.
Σ: Input alphabet: 47 symbols (ASCII letters, digits, '.', '-', ' ').
Γ: Output alphabet: 34 canonical tokens (e.g., JAVASCRIPT, REACT).
δ: Transition relation: Q × Σ → 2^Q (nondeterministic prefix tree).
ω: Emission function: ω(q, c) = ε for prefixes; ω(q_end, c) = K ∈ Γ.
q0: Unique start state shared across all word branches.
F: Set of accepting final states (|F| = 66 in M_SE, 55 in M_AI).

Prefix Transitions & Canonical Emission

```
δ(q0, 'J') = q_JS1  -->  ω = ε (silent prefix consumption)
δ(q_JS1, 'S') = q_JS2 ∈ F  -->  ω = JAVASCRIPT (emitted on final edge)
δ(q0, 'R') = q_React1  -->  ω = ε
δ(q_React1..5, 'e','a','c','t','.') = q_React6  -->  ω = ε
δ(q_React6, 'j') = q_React7  -->  ω = ε
δ(q_React7, 's') = q_React8 ∈ F  -->  ω = REACT (emitted on final edge)
```

Two Specialized Transducer Engines:

- M_SE: 438 states, 66 final states, 21 canonical software tokens.
- M_AI: 367 states, 55 final states, 17 canonical data & ML tokens.

CANONICAL TAXONOMY INTEGRITY

By separating software engineering (M_SE) and data/AI (M_AI) transducers, the system guarantees domain precision while unifying emissions into the 34-symbol canonical taxonomy.

Stage 2: Canonical Ordering Hierarchy for DFA Invariance

The Order Invariance Challenge

Candidates list qualifications in arbitrary physical layouts (e.g., Git before Python, or SQL before React). Naive finite automata would reject valid candidates due to permutations.

Canonical Sorting Function:

$$\text{Sort}(S, P) = \text{order by } (\text{Priority}(\text{Category}(s, P)), s) \quad \forall\, s \in (S \cap \Sigma_P)$$

1. Tokens outside profile alphabet Σ_P are discarded.
2. Tokens are ordered by tier priority, then alphabetically.
3. Guarantees deterministic input sequences into Stage 3 automata.

Step-by-Step Sorting Transformation

Raw Text Input:
`'Experienced in Git, Postgres, React.js, and JS'`

Stage 1 Extracted Tokens:
`['Git', 'Postgres', 'React.js', 'JS']`

Stage 2 Transduced Canonical:
`['GIT', 'POSTGRESQL', 'REACT', 'JAVASCRIPT']`

Full Stack Hierarchy Tiers:
`Language (1) -> Frontend (2) -> Backend (3) -> DB (4) -> Tools (5)`

Canonically Sorted Sequence:
`['JAVASCRIPT', 'REACT', 'POSTGRESQL', 'GIT']`

MATHEMATICAL RESULT

Permutation invariance is solved in O(N log N) canonical normalization, preserving strict linear DFA state-transition paths without combinatorial state explosion.

Stage 3: Profile Classification with Deterministic Finite Automata

Full Stack Developer

6 States · 1 Final State

`q0_start -> q1_lang -> q2_fe -> q3_be -> q4_db -> q5_final`

Enforces complete vertical competency: web language, frontend UI framework, backend server runtime, database, and version control tools.

Machine Learning Eng.

6 States · 1 Final State

`q0_start -> q1_lang -> q2_prep -> q3_framework -> q4_db -> q5_final`

Enforces core analytical pipeline: Python/R, data preparation (Pandas/NumPy), ML framework (PyTorch/Scikit-learn), storage, and tooling.

Backend Developer

4 States · 1 Final State

`q0_start -> q1_be_tech -> q2_db -> q3_tools`

Evaluates backend stack progression: runtime service framework (Node/Django/Spring), persistence engine, and infrastructure/Git.

Data Engineer

5 States · 2 Final States

`q0_start -> q1_role -> q2_core -> q3_proc -> q4_final`

Evaluates data pipeline stack: core programming (Python/SQL/Scala), distributed processing (Spark/Airflow), and analytical warehouses.

FORMAL GUARANTEE OF DETERMINISM

All profile recognizers are verified DFAs: $|\delta(q, a)| \leq 1$ for every symbol $a \in \Sigma$, with no ε-transitions. An absent transition terminates processing in immediate formal rejection.

Stage 3: Transition Structure & Acceptance Mechanics

Full Stack Acceptance Trace: Lina Marquez

- q0_start --> Consume 'JAVASCRIPT' --> q1_lang (Language verified)
- q1_lang --> Consume 'REACT' --> q2_fe (Frontend UI verified)
- q2_fe --> Consume 'NODE_JS' --> q3_be (Backend runtime verified)
- q3_be --> Consume 'POSTGRESQL' --> q4_db (Persistence verified)
- q4_db --> Consume 'GIT' --> q5_final ∈ F_FS (ACCEPTED!)

STATUS: ACCEPTED (String reaches accepting state q5_final)

ML Engineer Rejection Trace: Dario Pineda

- q0_start: Consume 'PYTHON' --> q1_lang (Language verified)
- q1_lang: Consume 'PANDAS' --> q2_prep (Data prep verified)
- q2_prep: Next token is 'POSTGRESQL' --> REJECTED: δ(q2_prep, POSTGRESQL) = ∅
- Fault Analysis: Missing ML Framework (Scikit-Learn/TensorFlow) --> Cannot advance to q3_framework
- Result: Cannot transition to q4_db or q5_final --> Candidate terminated before final state

STATUS: REJECTED (Missing prerequisite framework tier)

EXPLAINABLE FAILURE ANALYSIS

Unlike opaque ML confidence scores, ResumeLens pinpoints exactly WHICH token caused the rejection and WHICH tier state could not be reached, offering full mathematical transparency.

Stage 4: Context-Free Grammar (CFG) & textX DSL Specification

Formal CFG 4-Tuple Model

G = (V_N, V_T, P, S)

V_N (Non-Terminals):

{ ResumeModel, Candidate, ContactInfo, Skills, Evaluation, ProfileStatus }

V_T (Terminals):

{ 'candidate', 'skills', 'evaluation', 'ACCEPTED', 'REJECTED', STRING, INT }

P (Production Rules):

EBNF grammar defining structured candidate declarations and typing.

S (Start Symbol):

ResumeModel (top-level root of Abstract Syntax Tree)

Generated DSL Concrete Syntax

```
resume LinaMarquez {
  candidate {
    name: "Lina Marquez"
    email: "lina.marquez@example.com"
  }
  skills {
    canonical: [JAVASCRIPT, REACT, NODE_JS, POSTGRESQL, GIT]
  }
  evaluation {
    profile "Full Stack Developer": ACCEPTED
    profile "Backend Developer": ACCEPTED
    profile "Machine Learning Engineer": REJECTED
    profile "Data Engineer": REJECTED
  }
}
```

TYPING & INTEROPERABILITY

textX parses the DSL into a validated Python metamodel, ensuring candidate records conform to strict formal schema rules before report generation.

Stage 5: Multi-Format Reporting & Complete Audit Trail

HTML Visual Scorecard

Interactive Visual Dashboard

- Candidate header with verified contact badges.
- Transduced skill list with canonical tag representation.
- Acceptance/rejection breakdown for all 4 profiles.
- Auditable execution summary for recruiters and auditors.

Markdown Summary

Automated CI/CD Integration

- Lightweight plain-text summary format.
- Ideal for git-based recruitment workflows.
- Includes tabular evaluation summaries.
- Zero proprietary dependencies for viewing.

Canonical DSL Serializer

Typed Machine Interop

- Deterministic textual serialization.
- Validates 100% against the textX grammar.
- Enables downstream ATS system consumption.
- Idempotence: repeated runs yield identical files.

ONE-LINE CLI DEMO EXECUTION

`Command: python -m src.main --demo (Processes all 6 synthetic resumes, builds formal models, and emits all report formats in under 1.2 seconds).`

Experimental Evaluation: Synthetic Dataset Results

Resume File	Candidate Name	Target Profile	Observed Accepted Profiles
`fullstack_valid.txt`	Lina Marquez	Full Stack Developer	Full Stack Developer, Backend Developer
`fullstack_invalid.txt`	Mateo Rios	Full Stack Developer	Backend Developer (Full Stack rejected)
`backend_sample.txt`	Sara Beltran	Backend Developer	Backend Developer
`ml_engineer_valid.txt`	Celeste Arboleda	ML Engineer	ML Engineer, Backend Developer
`ml_engineer_invalid.txt`	Dario Pineda	ML Engineer	Backend Developer, Data Engineer (ML rejected)
`data_engineer_sample.txt`	Rafaela Quintero	Data Engineer	Data Engineer

EMPIRICAL FINDING: PROFILE OVERLAP

Notice that Full Stack and ML candidates frequently satisfy the Backend Developer automaton. This reflects real-world skill subsetting (Full Stack includes backend skills) rather than an error in classification.

Verification & Quality Assurance: 100/100 Tests Passing

100

Automated Tests Passing

- Suite runtime: 4.24 seconds
- Zero test failures or errors
- 100% deterministic suite
- Continuous integration ready

Test Coverage by Architectural Layer

- test_e2e_resumes.py (37 Tests): End-to-end dataset screening, cross-profile evaluation, output idempotence.
- test_extraction.py (15 Tests): Email, phone, LinkedIn, GitHub, and technical skill regex boundary checks.
- test_normalization.py (10 Tests): FST prefix transitions, emission verification, and alias resolution.
- test_visualization_html.py (10 Tests): HTML template rendering, candidate badge generation, report stability.
- test_recognition.py (9 Tests): Automata state acceptance, rejected paths, canonical order invariance.
- test_visualization_md.py (6 Tests): Markdown report generation, table formatting, and CLI summary blocks.
- test_taxonomy.py (5 Tests): Alphabet integrity, role priority configuration, and tier uniqueness.
- test_dsl.py & env (8 Tests): textX grammar validation, AST deserialization, and environment checks.

VERIFICATION RIGOR

Unit tests verify individual tuple properties (transitions, emissions, symbols); integration tests verify full pipeline idempotence (repeated executions yield bit-for-bit identical DSL outputs).

Conclusions, Formal Limitations & Future Research

Critical Appraisal & Limitations

Acceptance ≠ Candidate Competence:
ACCEPTED proves only that the extracted sequence matched the formal grammar and automaton. It does not measure real-world performance, coding skill, or absence of bias.

Synthetic Dataset Scope:
Tests were conducted on 6 synthetic resumes. Real-world resumes contain OCR errors, arbitrary PDF layouts, and unstructured narrative prose.

Lexical Horizon:
Currently bounded to 47 input symbols and 34 canonical skills. Skill variations outside the dictionary are omitted.

Future Work & Theoretical Extensions

Context-Sensitive Grammars (Type-1):
Model cross-field dependencies (e.g. binding years of experience directly to specific programming languages rather than global counters).

Multilingual Transducers:
Expand transducer alphabets to support Spanish, Portuguese, and international technical terminology.

Weighted Automata & Probabilistic FSTs:
Incorporate edge weights for candidate ranking while preserving the deterministic, inspectable transition structure.

REPOSITORY & OPEN RESEARCH

https://github.com/nicolasbuelvas/IntegrativeTask_AlbertoIA · Discrete Structures III · Thank You! Questions?